

Grand Oral

DVA: Pourquoi pas simplement le GPS et pourquoi 457kHz ?

$f = \frac{c}{\lambda} \Leftrightarrow \lambda = \frac{c}{f} = \frac{3 \cdot 10^8}{457 \cdot 10^3} \approx 656,5m \Rightarrow$ pas d'interférence avec flocons de neige car $a \leq \lambda$ pour avoir de l'interférence mais si $a \ll \lambda$, l'onde n'est pas impactée car:

Onde longue pour mieux éviter les obstacles (roches arbres + peu de réflexions parasites car faux signaux)

champ proche: $d < \lambda, \frac{\lambda}{2\pi} \approx 100m$

L'émetteur est une bobine parcourue par un courant alternatif. Elle génère un **champ magnétique oscillant**

neige isolant: diélectrique

conditions: Baryvox 2 - portée 70m

plusieurs personnes, direction

centaines d'heures d'autonomie

L'eau liquide (même en faible quantité dans la neige humide) possède une fréquence de relaxation qui absorbe l'énergie (effet micro-ondes). Le signal serait étouffé en quelques mètres.

Méca saut escalade (dyno) (rapport poids muscle (=carburant fusées))

$\vec{F} = \frac{d\vec{p}}{dt} = m\vec{a}$ car la masse est constante

$\vec{F} = m\vec{g}$, néglige frottements de l'air

Même vitesse au départ: v_i , mais en fonction de l'angle:

$$v_x = \cos(\alpha) * v_i$$

$$v_y = \sin(\alpha) * v_i$$

$$v_x^2 + v_y^2 = \cos^2(\alpha) * v_i^2 + \sin^2(\alpha) * v_i^2 = v_i^2$$

$$x_x = \cos(\alpha) * v_i * t + x_0 = \cos(\alpha) * v_i * t$$

$$x_y = \sin(\alpha) * v_i * t$$

Problème: pas très intéressant car il y a juste à sauter le plus collé au mur possible, peut être faudrait-il prendre en compte le volume de l'athlète ou autre.

rapport poids muscle:

On cherche à maximiser v_i , comment donner le plus de puissance possible ?

Déjà, $v_i = \int_0^{t_{\text{saut}}} a(t) dt$

⇒ maximiser t_{saut} , pour garder de l'accélération le plus longtemps possible

$a = \frac{F}{m}$, si on prend $F = k * m_{\text{muscle}}$ et $m = m_{\text{corps}} + m_{(\text{muscle})}$

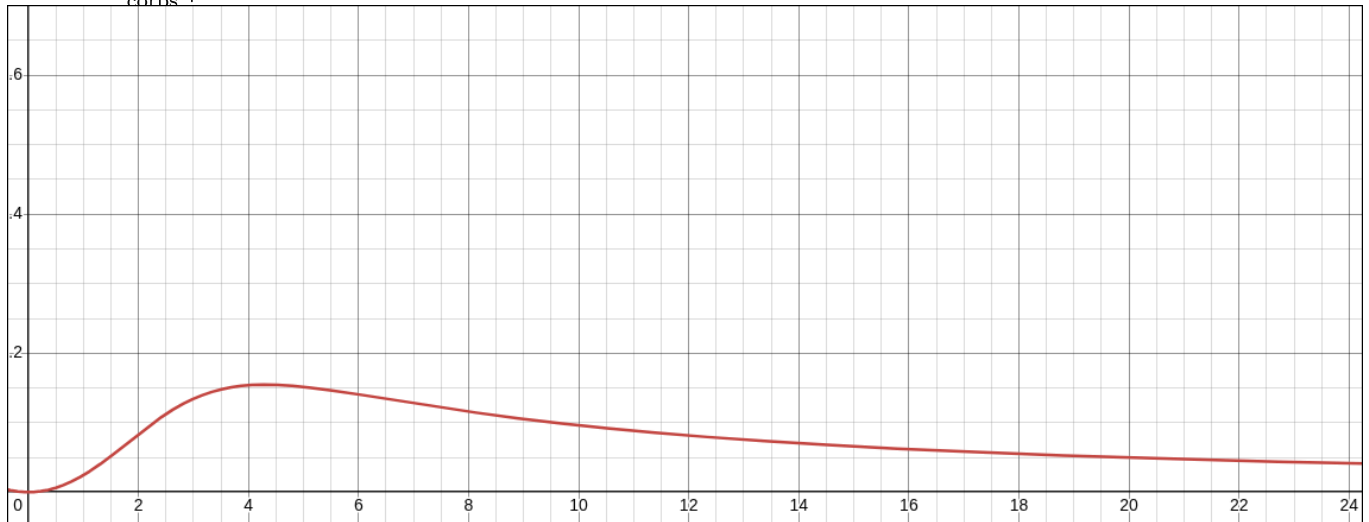
$a = k * \frac{m_{(\text{muscle})}}{m_{\text{corps sans muscle}} + m_{\text{muscle}}}$

Si on prend $m_{\text{corps}} = 50\text{kg}$, on peut tracer le graphique suivant:

Problème modélisation: grimpeurs sont très fins et ne cherchent pas à maximiser la masse musculaire à tout prix

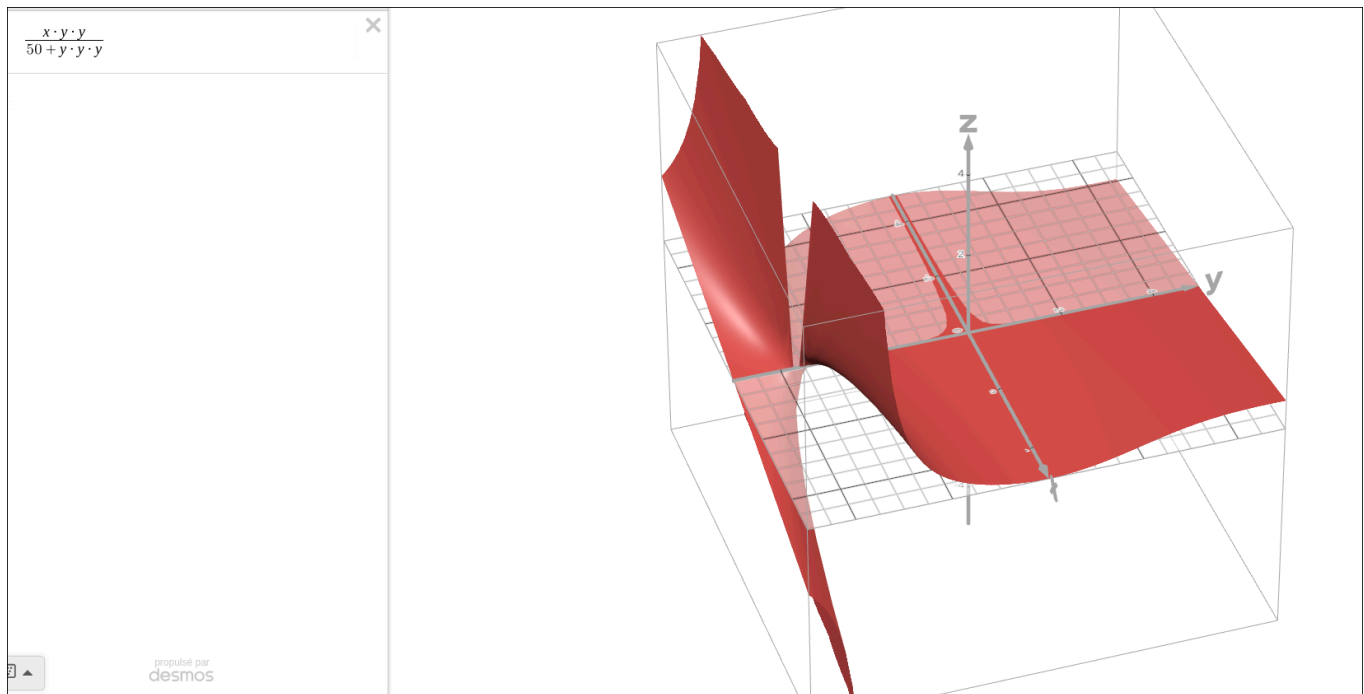
Mais la masse du muscle ne nous donne pas directement sa force ! $F = k * L^2$, car L la section du muscle, alors que sa masse est proportionnel à son volume L^3 :

$a = \frac{k * L^2}{m_{\text{corps}} + L^3}$



$L = \sqrt{m_{\text{muscle}}} = 2\text{kg}$

$x \cdot \frac{x}{k+x \cdot x \cdot x} \cdot 10$ avec $k = 50$ sur desmos



Différentes approches numériques de la suite de Fibonacci - Grand oral

Fait à l'aide de: [CP Algorithms - Fibonacci numbers](#)

Différentes manières de calculer la plus grandes valeur de Fibonacci, le plus rapidement possible sur ordinateur

1ère approche - version naïve

Voici la définition de la suite de Fibonacci:

$$F_0 = 0, F_1 = 1$$

$$F_n = F_{n-1} + F_{n-2}$$

Si on reprend cette définition et qu'on essaie de l'implémenter en Python:



Python

PYTHON

```
def fib_naive(x):  
    if x == 0 or x==1: return x  
    return fib(x-1)+fib(x-2)
```

Mais si on essaie de calculer une valeur relativement grande comme `fib(100)` par exemple, cela peut prendre énormément de temps !

<faire schéma arbre>, on pourrait réutiliser ces valeurs pour ne pas refaire les calculs bêtement (LookUp Table):

▶ ▼ 🗑

Python

PYTHON

```
lut = {
    0: 0,
    1: 1
}
def fib_lut(x):
    if lut.get(x) != None: return lut[x]
    r = fib(x-1)+fib(x-2)
    lut[x] = r
    return r
```

On peut aussi le calculer comme on ferait à la main:

▶ ▼ 🗑

Python

PYTHON

```
def fib_iter(x):
    a = 0
    b = 1
    for i in range(x):
        tmp = a + b
        a = b
        b = tmp
    return a;
```

On commence à arriver à de très grands nombres, si vous ne le faites pas déjà (python le fait très bien, avec pleins d'optimisations), l'algorithme de multiplication de Karatsuba peut nous aider de passer de la multiplication naïve $O(n^2)$ à $O(n^{\log(2)})$!

Multiplication de matrices

Essayons une autre approche:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} = \begin{pmatrix} F_n + F_{n-1} \\ F_n \end{pmatrix} = \begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix}$$

On peut donc multiplier des matrices pour calculer des éléments de la suite de Fibonacci !

On peut ensuite démontrer avec une preuve par récurrence <oral>:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} = \begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix}$$

Et puisque:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} F_2 & F_1 \\ F_1 & F_0 \end{pmatrix}$$

, on peut simplifier et partir directement de:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}$$

Mais l'intérêt n'est pas évident, en effet la multiplication de matrices semble bien plus long à calculer que de simples additions.

Mais la puissance possède une propriété intéressante: x^n avec

$$n = 2k \Rightarrow x^{2k} = x^{2^k}$$

On a du mal à voir l'intérêt, sauf si k lui-même est pair, donc: $k = 2k'$, dans ce cas:

$\Rightarrow x^n = x^{2^k} = x^{2^{2k'}}$. En bref, si on prend notre matrice, au carré, au carré, et ainsi de suite, on arrive à n en $O(\log_2(n))$ plutôt que $O(n)$.

On peut maintenant faire une implémentation simple:

(Je n'utilise pas numpy car il ne supporte pas des nombres de plus de 64 bits, or la suite de Fibonacci explose rapidement les compteurs)



Python

```

import math
class Mat:
    def __init__(self, vals):
        self.mat = vals
        self.shape = (2,2)
    def mul(self, rhs):
        a,b,c,d = self.mat
        e,f,g,h = rhs.mat
        return Mat([a*e+b*g, c*e+d*g, a*f+b*h, c*f+d*h])
    def exp(self, n):
        a = Mat(self.mat[:])
        for i in range(n):
            a = a.mul(self)
        return a
    def fast_exp(self, n):
        a = Mat(self.mat[:])
        base = Mat(self.mat[:])
        while n>0:
            if n&1:
                a = a.mul(base)
            base = base.mul(base)
            n >>= 1
        return a
def mat_fib(x): return Mat([1,1,1,0]).fast_exp(x).mat[3]

```

Faire de la vraie multiplication de matrices (donc plus que du 2×2) fonctionne très bien, mais je l'ai fait à la main pour réutiliser les valeurs, et c'est plus lisible.

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} * \begin{pmatrix} e & f \\ g & h \end{pmatrix} = \begin{pmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{pmatrix}$$

Formule explicite (Binet / De Moivre):

$$F_n = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}}$$

Mais on peut voir que le terme de gauche diminue très rapidement, donc on peut simplifier (pour de très grands nombres):

$$F_n = \left\lfloor \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n}{\sqrt{5}} \right\rfloor$$



Python

PYTHON

```
def explicit_fib(x):
    return int(((1+math.sqrt(5))/2)**x/(math.sqrt(5)))
```

Si on calcule l'erreur, elle peut être de plusieurs milliards ! Cela vient probablement de la précision de `math.sqrt(5)`.

Mais de toute manière, le pourcentage d'erreur reste faible (il diminue lorsque n augmente, et est proche de 10^{-14} pour des valeurs de $n = 10_{000}$)

Méthode de fast doubling

$$\begin{pmatrix} F_{2k+1} & F_{2k} \\ F_{2k} & F_{2k-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{2k} = \begin{pmatrix} F_{k+1} & F_k \\ F_k & F_{k-1} \end{pmatrix}^2$$

$$F_{2k+1} = F_{k+1}^2 + F_k^2$$

$$F_{2k} = F_k(F_{k+1} + F_{k-1}) = F_k(2F_{k+1} - F_k)$$



Python

PYTHON

```
def fib_log2(n):
    if n==0: return 0,1
    fst,snd = fib(n>>1)
    c = fst * (2 * snd - fst)
    d = fst**2+snd**2
    if n&1: return d,c+d
    return c,d
```